Effort-aware accuracy metrics:

How to evaluate machine learning classifiers in the supporting testing activity context

Davide Falessi

CONTEXT:

- ✓ Advances in defect prediction models (classifiers), have been validated via accuracy metrics.
- ✓ **Effort-aware metrics (EAMs)** relate to benefits provided by a classifier in accurately ranking defective entities such as classes or methods.
- ✓ The better the classifier, the higher the number of defective entities a developer can identify within a given rank.
- ✓ The different EAMs vary in **the thresholds** used to stop analyzing the ranking and **ranking criteria.**

AIM :

1) We reveal issues in EAMs usage, and
2) We propose and evaluate a normalization of PofBs (aka NPofBs), which is based on ranking defective entities by **predicted defect density**.

Example of EAM:
PofBx vs NPofBx (1):

PofBx is defined as the percentage of bugs that a developer can identify by inspecting the top x % of lines of code.

Example: PofB20

- The dataset is 1790 LOC,
- It features seven entities (three of which are
- 20% of the dataset would analyze only up to
- Ranked by probability of defectiveness,
- the first two entities,
- finding one-third of the defective entities,
- Thus, in this example, **PofB20 is 33%.**

PofB ranking			
ID	Size	Predicted probability	Actual
Entity 1	120	0.89	Defective
Entity 2	100	0.78	Not Defective
Entity 3	250	0.73	Defective
Entity 4	80	0.71	Defective
Entity 5	140	0.59	Not Defective
Entity 6	500	0.53	Not Defective
Entity 7	600	0.51	Not Defective

Table 1.1 Example of a dataset to compute an EAM metric called PofB20

EXAMPLE OF EAM: POFBX VS NPOFBX (2):

NPofBx: In order to improve the accuracy we suggest normalizing the PofBs. The idea behind normalization is that the user will follow a ranking based on the probability that an entity is defective, as provided by the classifier, normalized (i.e., divided) by the size of this entity.

Example: NPofB20

- The same dataset from **Table 1** (1790 LOC)
- 20% of the dataset would analyze only up to 358
- Ranked by predicted P /size,
- the first three entities,
- finding two-third of the defective entities,
- Thus, in this example, **NPofB20 is 66%.**

The normalization increased PofB20 from 33%

Therefore, normalizing PofB would lead to finding the double number of defect entities.

Normalized PofB ranking

ID	Size	Predicted probability	Predicted/Size	Actual
Entity 4	80	0.71	0.008875	Defective
Entity 2	100	0.78	0.0078	Not Defective
Entity 1	120	0.89	0.007416667	Defective
Entity 5	140	0.59	0.004214286	Not Defective
Entity 3	250	0.73	0.00292	Defective
Entity 6	500	0.53	0.00106	Not Defective
Entity 7	600	0.51	0.00085	Not Defective

Research Questions

– **RQ1**: Which EAMs are used in software engineering journal papers?

– **RQ2**: Does the normalization improve PofBs?

– **RQ3**: Does the ranking of classifiers change by normalizing PofBs?

– **RQ4**: Does the ranking of classifiers change across normalized PofBs?

RQ1: Which EAMs are used in software engineering journal papers?

Methodology RQ1

We perform a systematic mapping study featuring

Despite the importance of EAMs, there is no study investigating EAMs usage trends and validity. Therefore, in this thesis, we analyze trends in EAMs usage in the software engineering literature.

- **152 primary studies**
- **12 open-source projects**
- **10 EAMs**
- **10 classifiers**

Results:

RQ1: Which EAMs are used in software engineering journal papers?

Table 4.4 Number of EAM used in past studies.

# EAM	0	1	2	3	4	≥ 5
# PS	132	12	5	2	1	0

Our systematic mapping study reveals that the majority of studies used no EAM* and that most of those using EAMs use only a single EAM (e.g., PofB20).

**Ingored to validate the model according to their impact on effort.*

RQ2: Does the normalization improve PofBs?

RQ2 Methodology

- Since the testing effort varies according to the size of the entities under test, we had the intuition that the ranking of entities, is more effective if it takes into consideration both the **likelihood** of the entity to be defective and also its **size**.
- Specifically, we observe if the PofB of the same classifier on the same dataset increases after the normalization.
- **The independent variable**: the presence or absence of normalization in computing PofBs.
- **The dependent variable:** the score of PofB with and without the normalization.
- As PofBs we considered the spectrum from 10 to 90 with a step of 10 as well as the average (we neglected PofB0 & PofB100).
- Relative gain provided by normalization: $$\frac{(NPofB - PofB)}{PofB}$$

Results:

RQ2: Does the normalization improve PofBs?-*Measurement procedure*

1. **Perform Preprocessing**:
   - Normalize the data with log10.
   - Filter independent variables using correlation-based feature subset selection.
   - Apply SMOTE to balance each dataset.
2. **Create Train and Test Datasets**: Use the walk-forward validation technique

3. **Compute Predicted Probability:** Predict the probability of defectiveness for each class using each classifier.

4. **Compute Metrics**: Calculate PofBs and NPofBs.

Classifiers Used:

Decision Table
IBk (k-NN)
J48
KStar
Naive Bayes
SMO
Random Forest
Logistic
Regression
BayesNet
Bagging

Results:

RQ2: Does the normalization improve PofBs?

Table 4.6 Relative gain, statistical test and Cliff's delta results comparing a PofB metric with its normalization.

Compared Metrics	Relative Gain	Test Statistic S	Pvalue	Cliff's Delta
PofB10 vs. NPofB10	414%	31267	<0.0001	0.5804
PofB20 vs. NPofB20	224%	32089	<0.0001	0.5726
PofB30 vs. NPofB30	147%	33804	<0.0001	0.5588
PofB40 vs. NPofB40	107%	34122	<0.0001	0.5497
PofB50 vs. NPofB50	78%	34119	<0.0001	0.5217
PofB60 vs. NPofB60	51%	33533	<0.0001	0.4965
PofB70 vs. NPofB70	35%	33700	<0.0001	0.4695
PofB80 vs. NPofB80	23%	32107	<0.0001	0.4318
PofB90 vs. NPofB90	11%	31094	<0.0001	0.3540
Average_Pofb vs. NAverage_Pofb	38%	34446	<0.0001	0.5515

According to Table 4.6 the normalization significantly improves all ten PofB metrics.

**the effect size resulted as large for most EAMs (Cliff's delta value>=0.43 & pvalue<o.o5).*

RQ3: Does the ranking of classifiers change by normalizing PofBs?

RQ3: Methodolgoy

RQ3: Does the ranking of classifiers change by normalizing PofBs?

- **The dependent variable:** the rank of classifiers.
- **The independent variable**: the presence or absence of the PofB normalization.
- To compare the rankings we use the Spearman's rank correlation (Spearman, 1904) between the ranking of classifier provided by the same PofB, with versus without the normalization.

In order to interpret Spearman's values we used the following standard interpretation (Akoglu, 2018):

ρ Spearman's value	Interpretation
< 0.6	fair
< 0.8	moderate
< 0.9	very strong
$= 1$	perfect

- We also compare, for each dataset and PofB, if the best classifier coincides after the normalization.

Results:

RQ3: Does the ranking of classifiers change by normalizing PofBs?

Table 4.8 Proportion of times the same PofB metrics, with and without normalization, identifies the same classifier as best.

Compared metrics	Proportion of equivalent best classifiers
PofB10 vs. NPofB10	15%
PofB20 vs. NPofB20	12%
PofB30 vs. NPofB30	10%
PofB40 vs. NPofB40	12%
PofB50 vs. NPofB50	17%
PofB60 vs. NPofB60	15%
PofB70 vs. NPofB70	22%
PofB80 vs. NPofB80	27%
PofB90 vs. NPofB90	29%
Average_PofB vs. NAverage_PofB	5%

According to Table 4.8, in all ten PofBs the proportion of times that the same PofB metrics, with and without normalization, identifies the same classifier as best is less than half.
Thus, the best classifier is more likely to change than to coincide when considering PofB after the normalization.

RQ4: Does the ranking of classifiers change across normalized PofBs?

RQ4: Methodology

RQ4: Does the ranking of classifiers change across normalized PofBs?

- **The dependent variable:** the rank of classifiers.
- **The independent variable:** the *x* of the NPofB*x*, with *x* in the range 10 to 90 and its average.
- To compare the rankings we use the Spearman's rank correlation (Spearman, 1904) between pairs of NPofBs.
- We also compare for each dataset the proportion of ten NPofBs sharing the same classifier as best.

RQ4: Does the ranking of classifiers change across normalized PofBs?

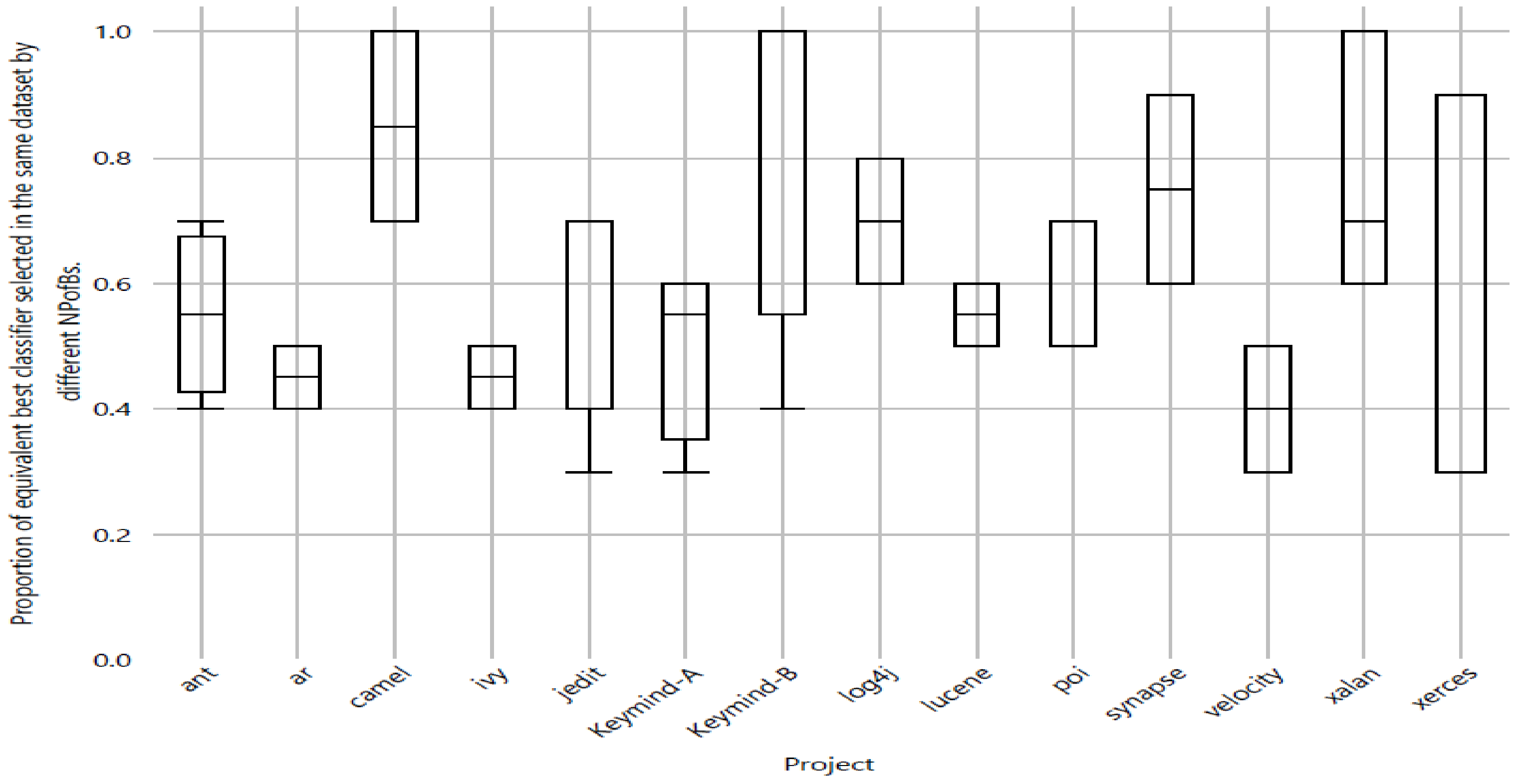


Fig. 4.5 Distribution of the proportion of datasets where the best classifier coincides across NPofBs.

The rankings of classifiers are far to be identical across different NPofBs.

According to Figure 4.5, in only five out of 41 datasets the best classifier for a dataset coincides across the ten PofBs. Thus, in about 88% of the cases, the best classifier varies across NPofBs.

Main results

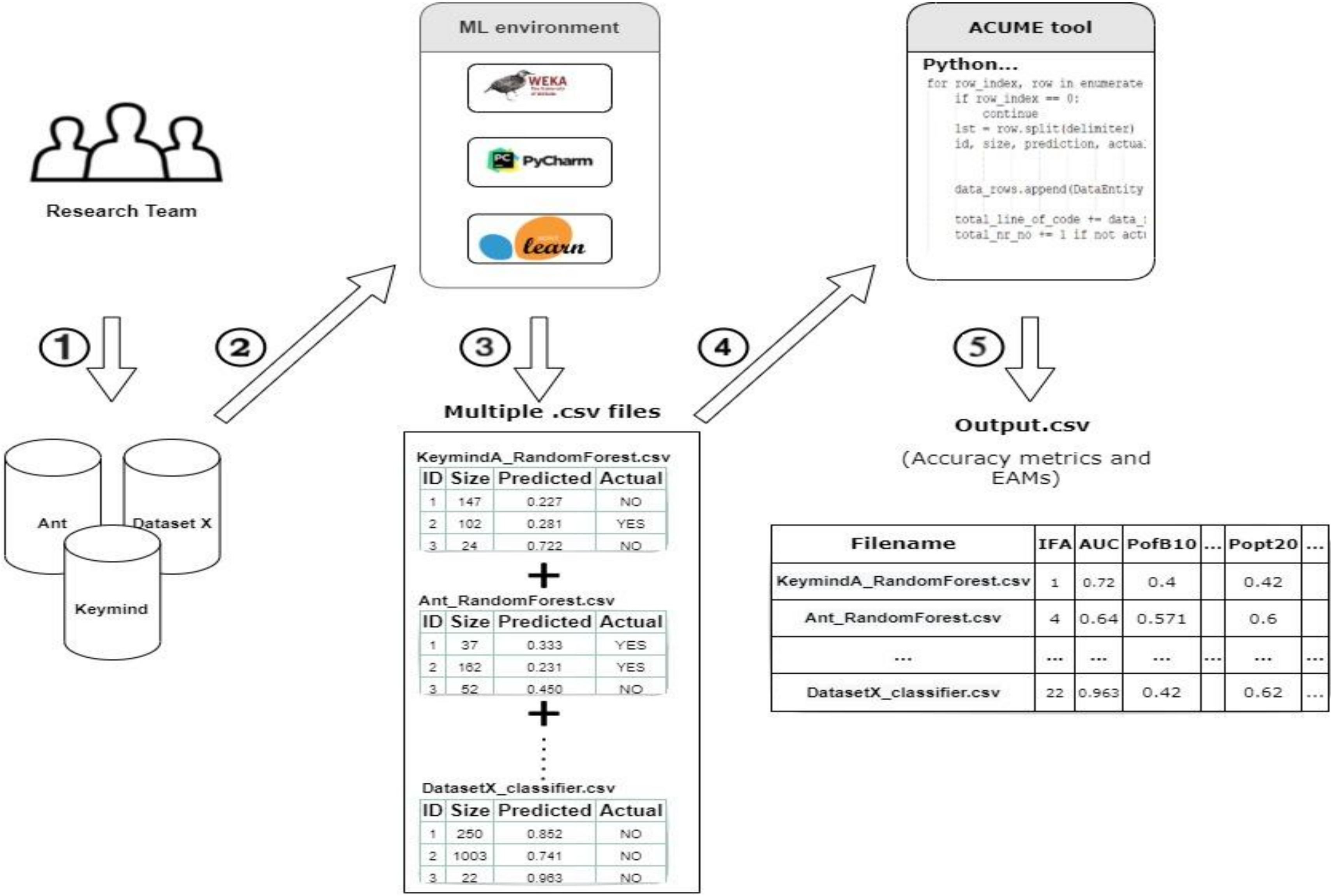
01 Our systematic mapping study reveals that most studies using EAMs use only a single EAM and that some studies mismatched EAMs names.

02 The main result of our empirical study is that NPofBs are statistically and by orders of magnitude higher than PofBs.

03 The best classifier is more likely to change than to coincide when considering PofB after the normalization.

04 The rankings of classifiers are far to be identical across different NPofBs.

We provide a tool to compute EAMs to support researchers in avoiding past issues in using EAMs.



ACUME Tool

References

- Çarka, J., Esposito, M. & Falessi, D. On effort-aware metrics for defect prediction. Empir Software Eng 27, 152 (2022). https://doi.org/10.1007/s10664-022-10186-7